

Computer Vision

Lab02: Feature extraction and matching

Ziyu Ma

Task1: Harris Corner Detection (relevant file: "extract_harris.py")

Step 1: compute image gradients in x and y direction.

```
Ix = signal.convolve2d(img, np.array([-1, 0, 1]).reshape(1, -1) * 0.5, mode="same")
Iy = signal.convolve2d(img, np.array([-1, 0, 1]).reshape(-1, 1) * 0.5, mode="same")
```

For this step, `scipy.signal.convolved2d` is used. The mode "same" is chosen to keep the image size unchanged.

Step 2: blur the image and compute the M matrix

```
Ixx = Ix * Ix
Iyy = Iy * Iy
Ixy = Ix * Iy

Mxx = cv.GaussianBlur(Ixx, (7, 7), sigma,
borderType=cv.BORDER_REPLICATE)
Myy = cv.GaussianBlur(Iyy, (7, 7), sigma,
borderType=cv.BORDER_REPLICATE)
Mxy = cv.GaussianBlur(Ixy, (7, 7), sigma,
borderType=cv.BORDER_REPLICATE)

M = np.array([[Mxx, Mxy], [Mxy, Myy]])
```

According to the formula of M in the exercise slide of step2. `Ixx`, `Iyy`, `Ixy` are computed. Then `cv.GaussianBlur` is used for gaussian filtering. Then the blurred parts are composed to build the auto-correlation matrix M.

Step 3: compute Harris response

```
det = Mxx * Myy - Mxy * Mxy
trace = Mxx + Myy
C = det - k * (trace ** 2)
```

This step uses the formula $R = \det(M) - k \cdot \text{trace}(M)^2$.

Step 4: non-maximum suppression

```
condition1 = C > thresh
```

```

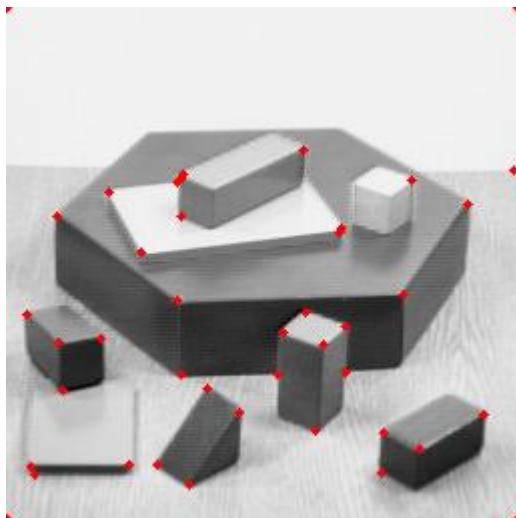
C_max_filter = ndimage.maximum_filter(C, 3)
condition2 = (C == C_max_filter)

corners = np.argwhere((condition1 & condition2))
corners[:, [0,1]] = corners[:, [1,0]]

```

Two conditions must be met to decide if a pixel is a corner. Its response function value must be beyond a threshold, and it must have the maximum value in its neighborhood. `scipy.ndimage.maximum_filter` is used for non-maximum suppression.

Result demonstration:



“blocks.png”



“house.png”

Task2: Description and Matching (relevant files: “extract_descriptors.py” and “match_descriptors.py”)

Implement filter_keypoints function in “extract_descriptors.py”

```

img_h, img_w = img.shape[0], img.shape[1]
d = patch_size // 2

x_coord = keypoints[:,0].flatten()
y_coord = keypoints[:,1].flatten()

x_filter1 = x_coord >= d
x_filter2 = x_coord <= img_w - d
y_filter1 = y_coord >= d
y_filter2 = y_coord <= img_h - d

x_filter = np.logical_and(x_filter1, x_filter2)
y_filter = np.logical_and(y_filter1, y_filter2)
total_filter = np.logical_and(x_filter, y_filter)

```

```
return keypoints[total_filter]
```

This function filters out keypoints that are too close to the edges. Say $d = \text{patch_size} // 2$. Only keypoints with x coordinate inside $[d, \text{img_w} - d]$ and y coordinate inside $[d, \text{img_h} - d]$ are kept in the keypoints array.

Implement functions of “match_descriptors.py”

```
desc1_reshape = np.reshape(desc1, (-1, 1, desc1.shape[-1]))
desc2_reshape = np.reshape(desc2, (1, -1, desc2.shape[-1]))

distances = np.sum((desc1_reshape - desc2_reshape) ** 2, axis=-1)
```

The above code is for calculating the sum of squared differences between the images' descriptors.

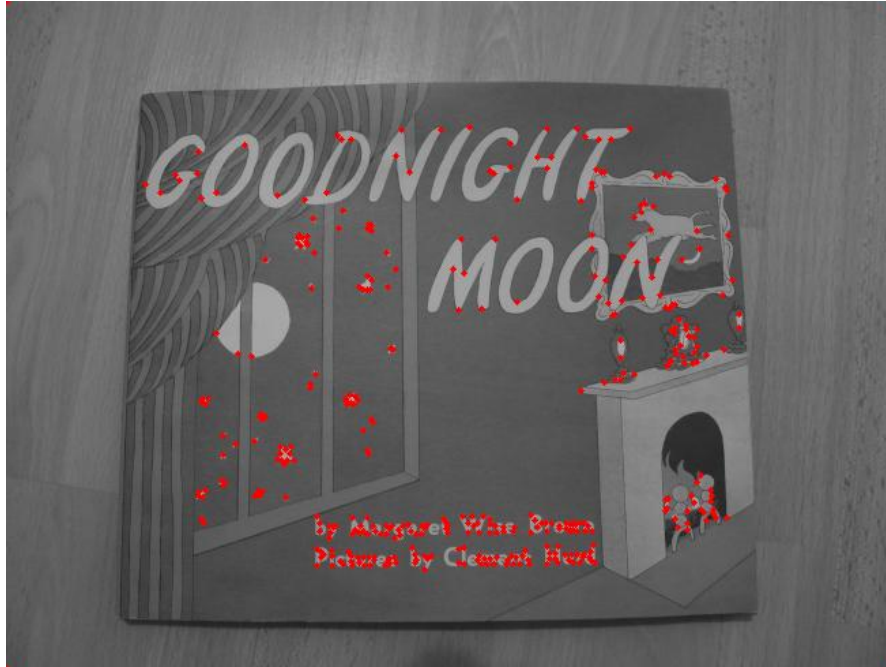
```
indices_c = np.argmin(distances, axis=1)
row_num = np.arange(desc1.shape[0])
matches_forward = np.column_stack((row_num, indices_c))

if method == "one_way": # Query the nearest neighbor for each
keypoint in image 1
    # TODO: implement the one-way nearest neighbor matching here
    # You may refer to np.argmin to find the index of the minimum
over any axis
    matches = matches_forward
elif method == "mutual":
    # TODO: implement the mutual nearest neighbor matching here
    # You may refer to np.min to find the minimum over any axis
    indices_r = np.argmin(distances, axis=0)
    col_num = np.arange(desc2.shape[0])
    matches_backward = np.column_stack((indices_r, col_num))
    matches = np.array([x for x in set(tuple(x) for x in
matches_forward) & set(tuple(x) for x in matches_backward)])
elif method == "ratio":
    # TODO: implement the ratio test matching here
    # You may use np.partition(distances,2,axis=0)[: ,1] to find the
second smallest value over a row
    two_least_distant = np.partition(distances, 2)[: ,:2]
    ratio_check = two_least_distant[: ,0] / two_least_distant[: ,1] <
ratio_thresh
    matches = matches_forward[ratio_check]
else:
    raise NotImplementedError
```

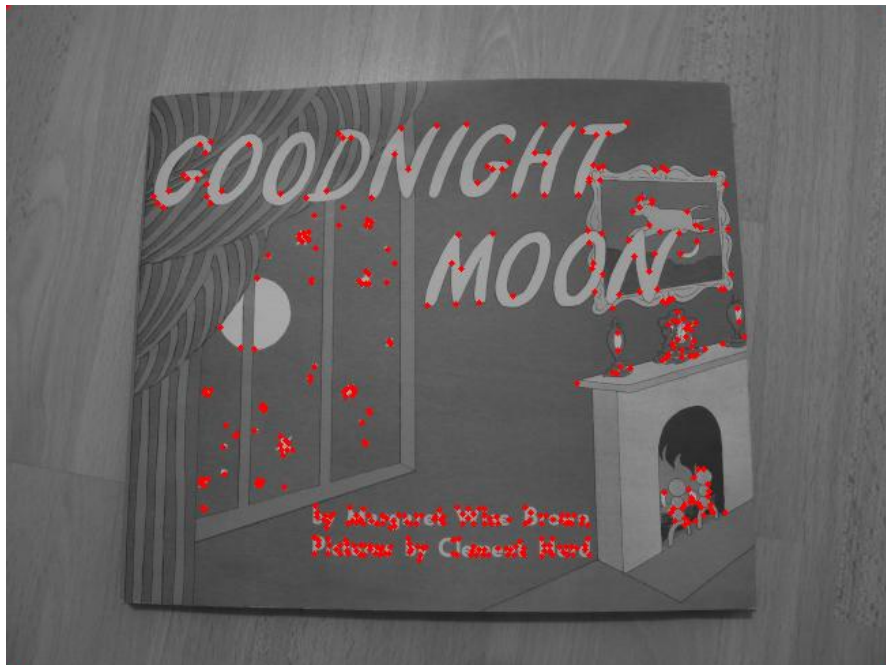
Three different matching methods are implemented. For one-way nearest neighbors matching, each feature from the img1 is matched to its closest feature from img2. For mutual nearest neighbors matching, a matching (a, b) is accepted only when b is the closest feature of img2 for a and a is the closest feature of img1 for b. For ratio test matching, only matchings with the

ratio between the 1st and the 2nd nearest neighbor lower than a threshold are accepted.

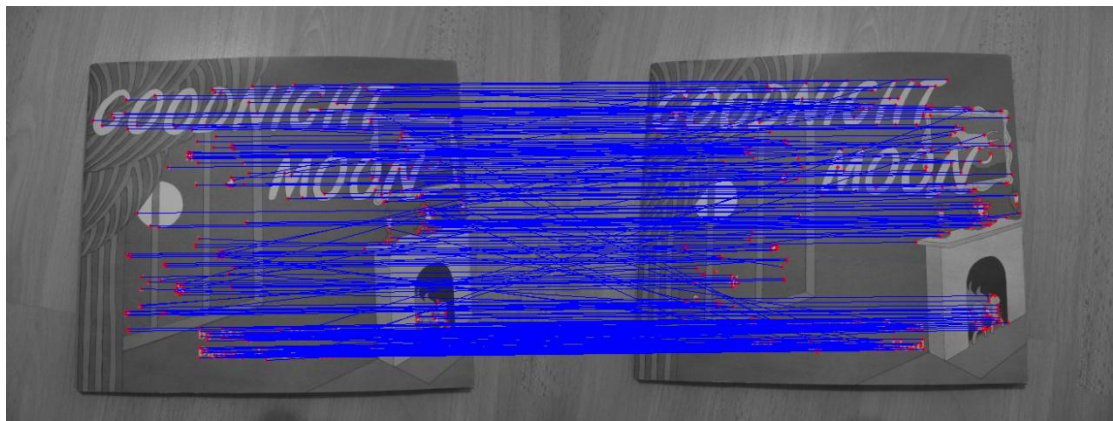
Result demonstration:



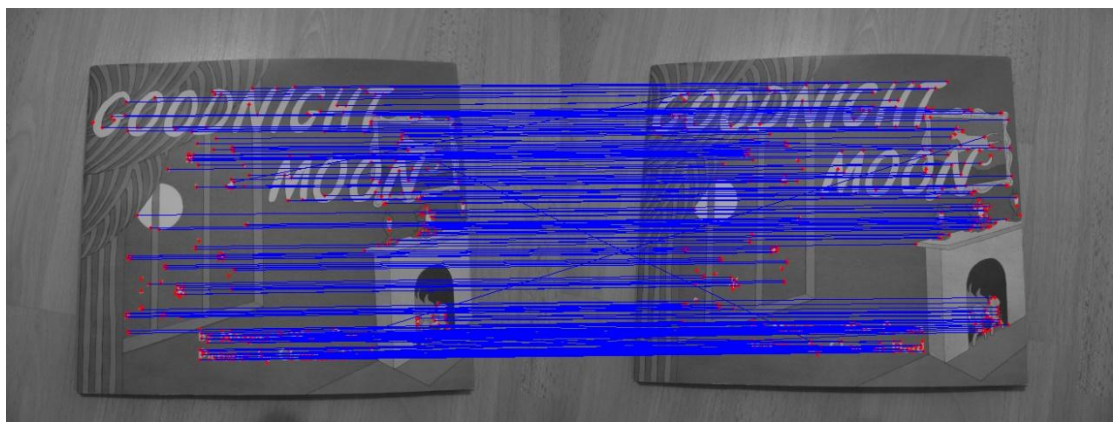
"l1_harris.png"



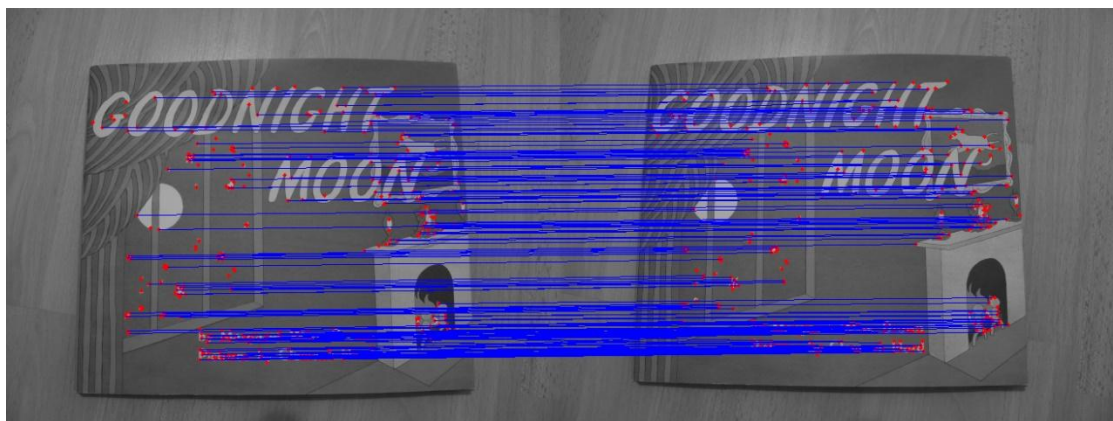
"l2_harris.png"



“match_ow.png”



“match_mutual.png”



“match_ratio.png”

Ordering of result (better to worse): ratio>mutual>one-way.